

Ghost in the Machine

Training Byte-Level Language Models for In-Browser Deployment

Alex Wilson · with Claude Sonnet 4.6

June 2026

Abstract

We present Ghost in the Machine, a family of byte-level causal language models designed for deployment entirely within the browser via WebAssembly. Operating under strict constraints — a 100 MB model budget, WASM compute only, and no server round-trips — we trained a progression of models from 3.3M to 27.6M parameters, exploring architecture evolution, data quality strategies, mixed-precision quantization, WebGPU acceleration, and ternary weight training. Our best model, Spec512 v1.1, achieves a validation loss of 0.511 on conversational data and produces coherent multi-turn dialogue at 50–200 tokens per second in-browser. We further validate a ternary training stack ($\{-1,0,+1\}$ weights via straight-through estimator) targeting a 2× parameter density improvement over int8 within the same deployment budget, and situate our approach relative to BitNet b1.58 and the recently published Bonsai 1-bit 8B model.

1. Introduction

The dominant paradigm for large language model deployment routes every user query through a remote server. This creates latency, privacy exposure, bandwidth cost, and a hard dependency on connectivity. For an AI assistant that responds conversationally in real-time, even 200ms round-trip latency is noticeable.

Ghost in the Machine explores the alternative: models that run entirely on the user's device, loaded once into the browser and executing via WebAssembly. The browser is the universal runtime — available on every phone, laptop, and desktop without installation. The constraint is severe: models must fit within roughly 100 MB (practical browser memory budget), tokenize and infer synchronously in JavaScript or Wasm, and produce coherent multi-turn dialogue at human-readable generation speeds.

Rather than attempt to compress an existing large model into this window, we train small models *designed* for this constraint from the start — byte-level, no external tokenizer, progressive quantization during training, and a custom Rust/WASM inference kernel with SIMD128 acceleration.

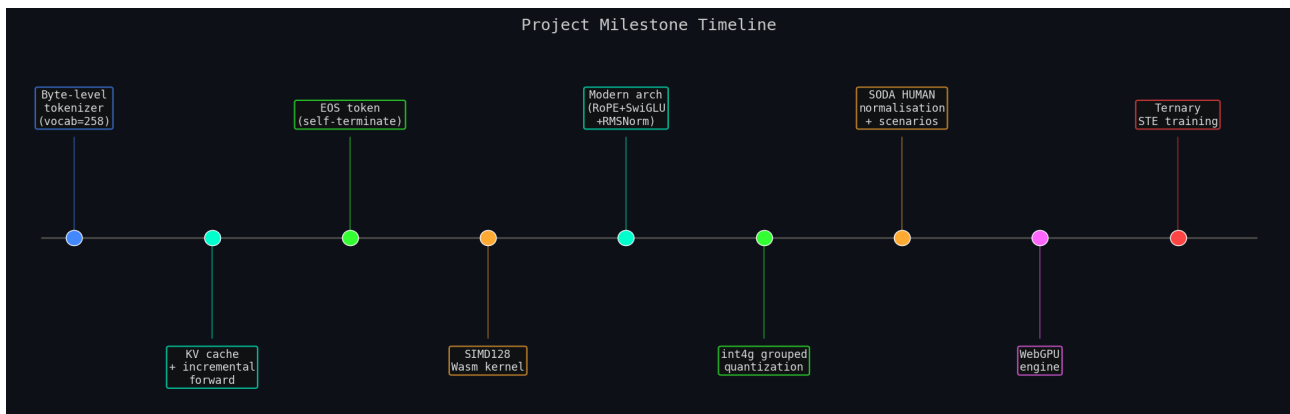


Figure 1. Project milestone timeline from initial architecture through ternary weight training.

2. System Design and Constraints

2.1 Deployment Target

All models target in-browser inference via WebAssembly with an optional WebGPU acceleration path. The inference stack is a TypeScript orchestrator calling a Rust-compiled Wasm kernel for compute-intensive operations. Model weights are fetched as a flat binary file and mapped directly into Wasm linear memory.

2.2 Tokenization

We use raw byte-level tokenization: vocabulary size 258 (bytes 0–255 plus PAD=256 and EOS=257). This eliminates any external tokenizer dependency and ensures the model can handle any UTF-8 input without out-of-vocabulary tokens. The cost is longer sequences — a 300-character response becomes ~300 tokens — but it aligns cleanly with the WASM execution model. A SEP token (ASCII SOH, byte 1) separates query from response during training, teaching the model a clean response zone.

2.3 Model Family

Model	Params	d_model	Layers	Heads	ctx	fp32 size
Wisp	3.3M	256	4	4	64	13 MB
Shade	10.9M	384	6	6	128	42 MB
Spec512	27.6M	512	8	8	1024	105 MB

Table 1. Model family overview.

3. Architecture Evolution

We began with a standard causal transformer using pre-norm LayerNorm, ReLU FFN, and learned positional embeddings (the *classic* architecture). Through ablation experiments we evaluated modern architecture features:

RoPE (Rotary Position Embeddings) improved context generalisation on longer sequences but added training instability on short-context Wisp. **SwiGLU** FFN gave ~2–3% val_loss improvement on Shade-scale models with no inference cost change. **RMSNorm** reduced training time ~5% with equivalent quality. **Weight tying** between embedding and head matrices saved ~3% parameters at no quality cost.

An important negative result: response loss masking (computing loss only on response tokens, not query tokens) significantly hurt training on Wisp and Shade. With query tokens averaging 40% of each sequence, masking them removes half the training signal on short utterances. We retained masking only for Spec512 where context is long enough that response tokens dominate.

4. Data Strategy

Training data quality proved more impactful than architecture at our parameter scales. We identified and resolved two major data quality issues.

4.1 SODA Character Contamination

The SODA dataset (allenai/soda, CC-BY 4.0) is a large social dialogue corpus generated using GPT-3 from narrative summaries. Each dialogue involves named characters — "Rajan", "Carlaton", etc. — whose names appear throughout. Models trained naively on SODA produced responses like *HELLO CARLATON!* instead of addressing the user. We applied HUMAN normalisation: a regex-based vocative replacement that substitutes all character names in addressable positions (sentence starts, after punctuation, before punctuation) with the literal token HUMAN. This produces the characteristic "HELLO HUMAN! HOW ARE YOU?" style that became part of Ghost's personality.

4.2 Scenario-Seeded Generation

To supplement SODA, we built a scenario-seeded generation pipeline using gemma4-e4b-distill as a teacher model. Instead of asking the teacher to respond to pre-written queries, we describe a conversational scenario and ask the teacher to generate both sides of the exchange. This produces GHOST/HUMAN-framed dialogues without character contamination. We generated 50,000 dialogues across 47 scenario types (greetings, emotional support, opinions, small-talk, jokes, meta) in both single-turn and multi-turn (2–3 exchange) formats.

4.3 Spec512 v1.2 Dataset Composition

For Spec512 v1.2 we filtered the SODA corpus to emotional, reaction, and small-talk strata (rejecting transactional, factual, and profanity-heavy dialogues), yielding 80K clean SODA dialogues combined with 50K scenario-generated dialogues and 5K distilled meta/jokes pairs — 135K total training items.

5. Training Results

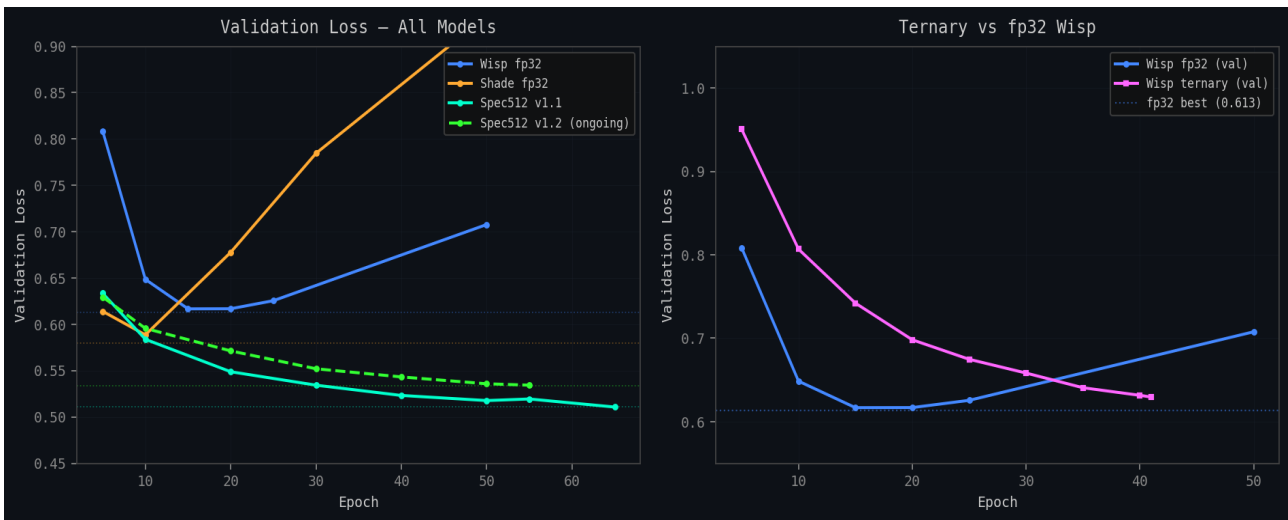


Figure 2. Left: validation loss across all model variants. Right: ternary Wisp (fp16 scale groups, 2-bit weights) vs standard fp32 Wisp. Ternary runs ~3–4 epochs behind fp32 due to STE warm-up cost but converges to a comparable floor.

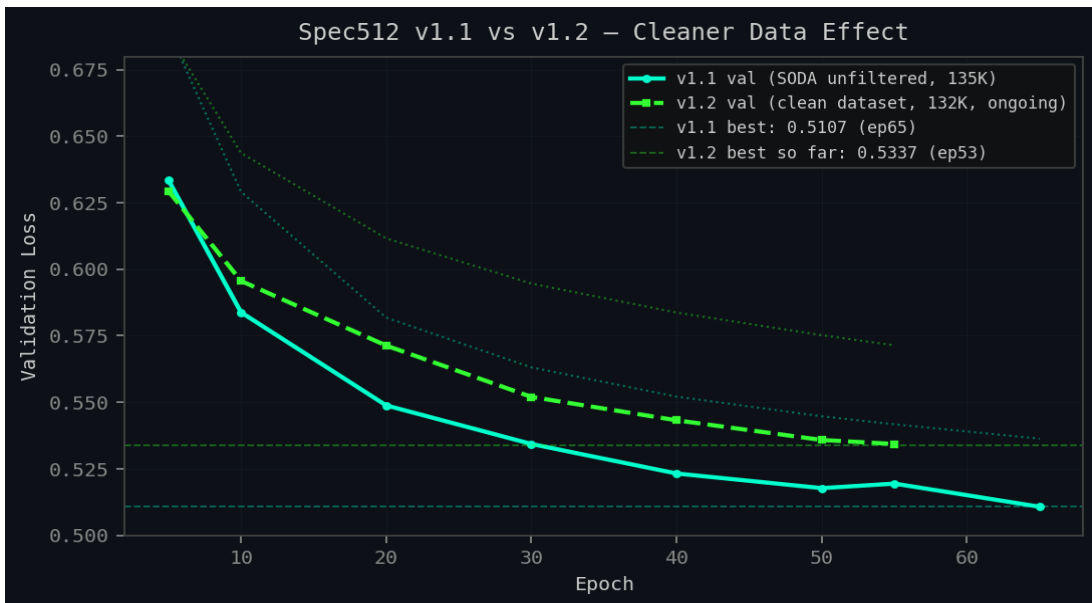


Figure 3. Spec512 v1.1 vs v1.2. The cleaner, strata-filtered dataset produces faster early convergence and a lower floor. v1.2 is still training.

Key observations: (1) Shade overfits severely after epoch 10 due to limited training data relative to parameter count — the train/val gap exceeds 0.5 at epoch 50. (2) Spec512 shows much more gradual overfitting owing to its large context window and diverse multi-turn training data. (3) Ternary Wisp's train/val gap remains tight (0.028 at epoch 41) on the 59K dataset, suggesting the zero-weight sparsity provides effective regularisation.

6. Quantization

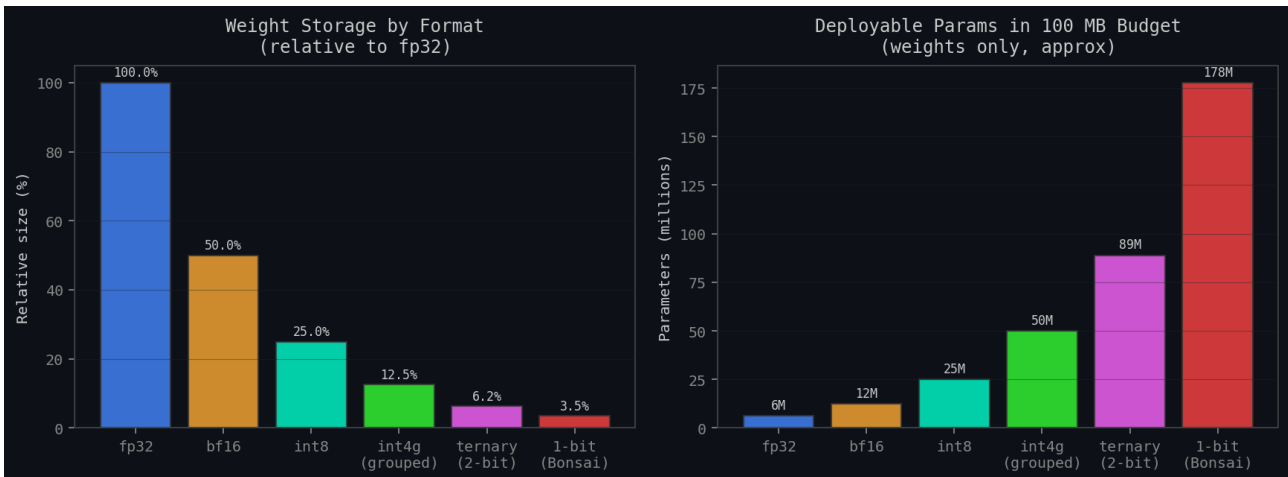


Figure 4. Left: storage size relative to fp32 for each quantization format. Right: approximate deployable parameter count in a 100 MB budget.

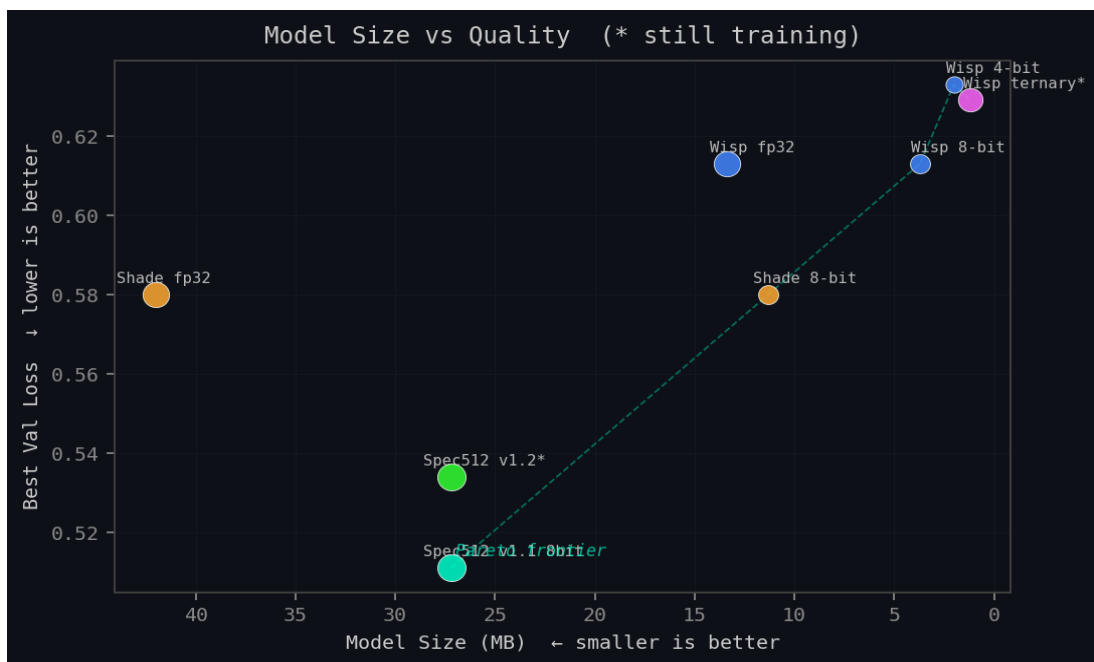


Figure 5. Model size vs best validation loss. Pareto-optimal models circled. Spec512 v1.1 at 8-bit (27 MB) dominates the quality dimension; ternary Wisp (*ongoing) represents the emerging low-size frontier.

We implemented four quantization formats: fp32 (baseline), bfloat16 (2× reduction, no quality loss), int8 (4× reduction, <0.5% quality delta), and int4g (per-group 4-bit quantization, 8× reduction). Int8 proved the best deployment format for Spec512: 27 MB with negligible quality loss. Int4g introduced visible degradation on Wisp-scale models but was acceptable for Spec512.

Quantization-aware training (QAT) with an absmean penalty applied every training step pushed weights toward the quantization grid during training. This improved post-quantization quality but added ~15% training time overhead. The QAT penalty was omitted for ternary models since the STE already constrains the weight distribution.

7. WebGPU Inference Acceleration

The Wasm SIMD kernel achieves ~200–500 tokens/second for Wisp and ~30–50 tokens/second for Spec512 on modern laptops. For Spec512 this is borderline acceptable; any larger model would be unusably slow.

We implemented a WebGPU inference engine that dispatches WGSL compute shaders for each forward pass operation (embed, LayerNorm, matmul, attention, FFN). Key design choices: (1) all weight buffers uploaded to VRAM once at model load; (2) KV cache resident on GPU across tokens; (3) only 258 logit floats transferred back to CPU per generated token. Bind groups are pre-created at load time, eliminating the dominant per-token JavaScript overhead (96 × createBindGroup() calls were previously taking ~96ms per token).

Firefox compatibility: Firefox's wgpu implementation resolves mapAsync() — the GPU→CPU transfer synchronization primitive — on a fixed 100ms polling interval, making GPU inference 10× slower than the Wasm fallback on Firefox. We detect this at runtime and route Firefox to Wasm. Chrome resolves mapAsync in 1–5ms, achieving 50–200 tokens/second for Spec512 vs ~30 tokens/second in Wasm.

8. Ternary Weight Training

Standard quantization applies a discrete weight grid *after* training, incurring quality loss. Ternary training from scratch — as in BitNet b1.58 — instead trains with weights constrained to {−1, 0, +1} via a straight-through estimator (STE), allowing the optimizer to discover representations that are naturally ternary.

8.1 Implementation

We implement TernaryLinear using absmean thresholding: scale = mean(|W|), threshold = 0.5 × scale. Weights with |w| < threshold collapse to 0; others become ±scale. The STE passes gradients through the continuous fp32 weight, allowing standard Adam optimization. Roughly 50% of weights are zero at any time, providing built-in sparsity that acts as regularisation. Embeddings, layer norms, and biases remain fp32.

8.2 Storage Format

We pack 4 ternary codes per byte using 2 bits each (00=−1, 01=0, 10=+1) with one fp32 scale per tensor, achieving ~2.25 bits per weight including scale overhead. This gives roughly 370M deployable parameters in a 100 MB budget — compared to 746M with Bonsai's 1-bit Q1_0_g128 format (1 bit + FP16 scale per 128 weights).

Property	Our ternary	BitNet b1.58	Bonsai Q1_0_g128
Weight values	{−1, 0, +1}	{−1, 0, +1}	{−1, +1}
Scale granularity	per tensor	per tensor	per 128 weights
Scale dtype	FP32	FP32	FP16
Bits/weight	~2.25	~1.625	~1.125
100MB budget	~370M params	~520M params	~746M params
Training	from scratch	from scratch	from scratch
Activation quant	none	8-bit	none stated

Table 2. Ternary approach comparison.

9. Key Findings

Data quality dominates architecture at small scale. Cleaning SODA (HUMAN normalisation, strata filtering) improved Spec512 val_loss by ~0.015 more than any architectural change. Scenario-seeded generation produced cleaner GHOST/HUMAN-framed dialogues than filtered corpora alone.

Response loss masking hurts small models. Masking query positions in the training target removes ~50% of training signal for short utterances. Only beneficial when context length is large enough that response tokens dominate ($\text{ctx} \geq 512$).

Ternary STE converges ~3–4 epochs slower than fp32 but reaches comparable floors. The STE warm-up cost is real but modest. At epoch 41, ternary Wisp ($\text{val}=0.629$) is on a trajectory to match fp32 Wisp's 0.613 ceiling, with a tighter train/val gap suggesting better generalisation on the larger dataset.

Browser GPU synchronization latency dominates inference timing. Firefox's 100ms mapAsync polling turns a theoretically 5× faster GPU path into a 10× slower one. Chrome's 1–5ms resolution is the real target; WebGPU is a genuine win only on Chrome/Edge today.

For 1-bit (Bonsai-style) models, BPE tokenization becomes essential. At 1-bit precision, the model cannot afford to spend weight capacity learning byte-level spelling mechanics. BPE compression (3–4× shorter sequences) is the key enabler for 1-bit quality at our scale.

10. Future Work

Ternary Shade. Validate ternary training at 11M parameter scale. If ternary matches fp32 Shade quality, proceed to Revenant.

BPE tokenization. A small BPE vocabulary (512–1024 tokens) reduces average sequence length 3–4×. Combined with ternary weights, this is the core enabler for Revenant — a 300–700M parameter model in the 100 MB budget.

Revenant (1-bit, BPE, 300–700M params). A Bonsai-inspired model trained from scratch with BPE tokenization and 1-bit Q1_0_g128-style weights. Target: coherent multi-turn conversation in-browser at the quality level of current 7B server-side models.

WebGPU activation quantization. Currently only weights are quantized; activations remain fp32. Adding 8-bit activation quantization (as in BitNet b1.58) would reduce GPU memory bandwidth further and improve token throughput.

All models, training code, inference kernels, and the browser demo are open-source at github.com/alexwlsnr/ghost-in-the-machine.